README FILE

Creating local server:

- First off, we need to make a local server.
- When you have opened pgAdmin, right click on servers, then go to register and then server.
- Once the menu opens, under General you want to name your server. I named mine Football_league. Then on that same screen go to Connection and type “localhost” into Host name/address. You will then need to enter your password in that same menu; this will be the password you created while setting up pgAdmin.

Creating Database:

- You will now see your server appear on the left-hand side when you click into servers.
- Click the drop-down menu for your server and you should see Databases.
- Right click this and press create.
- This is where you can name your database, I called mine “Football_database”.
- Now we have our database we need to put something in it.
- Right click on your created database and press Query Tool.
- This is where you need to paste the following code:

```
-- Creating Team Table
CREATE TABLE Team (
  team_id TEXT NOT NULL PRIMARY KEY,
  team_name TEXT NOT NULL
);

-- Creating Group Table
CREATE TABLE "group" (
  group_id TEXT NOT NULL PRIMARY KEY,
  group_name TEXT NOT NULL
);
```

```
-- Creating Competition Table
CREATE TABLE Competition (
   competition_id TEXT NOT NULL PRIMARY KEY,
   competition_type TEXT NOT NULL
);

-- Creating Venue Table
CREATE TABLE Venue (
   venue_id TEXT NOT NULL PRIMARY KEY,
   venue_name TEXT NOT NULL
);

-- Creating Schedule Table
CREATE TABLE Schedule (
   schedule_id TEXT NOT NULL PRIMARY KEY,
   match_date DATE NOT NULL,
   match_time TIME NOT NULL,
   match_week INT
);

-- Creating Player Table
CREATE TABLE Player (
   player_id TEXT NOT NULL PRIMARY KEY,
   player_name TEXT NOT NULL,
   team_id TEXT NOT NULL REFERENCES Team(team_id),
   group_id TEXT NOT NULL REFERENCES "group"(group_id),
   consultant_id INT NOT NULL
);

-- Creating Match Table
CREATE TABLE Match (
   match_id TEXT NOT NULL PRIMARY KEY,
   schedule_id TEXT NOT NULL REFERENCES Schedule(schedule_id),
   home_team_id TEXT NOT NULL REFERENCES Team(team_id),
   away_team_id TEXT NOT NULL REFERENCES Team(team_id),
```

```
  competition_id TEXT NOT NULL REFERENCES
Competition(competition_id),
  venue_id TEXT NOT NULL REFERENCES Venue(venue_id),
  result TEXT
);

-- Creating Event Type Table
CREATE TABLE Event_Type (
  event_type_id TEXT NOT NULL PRIMARY KEY,
  event_type TEXT NOT NULL
);

-- Creating Match Event Table
CREATE TABLE Match_Event (
  event_id TEXT NOT NULL PRIMARY KEY,
  match_id TEXT NOT NULL REFERENCES Match(match_id),
  player_id TEXT NOT NULL REFERENCES Player(player_id),
  event_type_id TEXT NOT NULL REFERENCES
Event_Type(event_type_id),
  minute INT NOT NULL CHECK (minute>0)
);

/*Only columns that allow null values are the result column in the
match table
and the week column in the schedule table*/
/*The minute column in the match event table must be greater than 0
as when
storing football events, the minute can only be 1 and above*/
```

END OF SCRIPT

- This script created empty tables that look like those from the excel file.
- To see that these tables have been created go to the drop-down menu for database, then schemas, then public, then tables which shows you the tables in your database. If they do not show straight

away, you can right click on Tables and refresh and they should come up.

Inserting Data

- To insert the data, you will need to download the csv files from my folder where you got this README file.
- They are all appropriately named for each table.
- PLEASE before you try and import them in PostgreSQL you will need to go into the schedule csv and change the Match_Date column from dd/mm/yyyy to yyyy-mm-dd and then save the file. For some reason even when you change that column in excel it does not change in the csv. If you make a mistake you might need to try it again this is the only error, I ran into.
- After you have done this, you can import the data.
- Go to the Tables menu again and right click on the table you want to import data to. You must follow the order in which the tables were created to ensure primary keys have data before they are needed for foreign keys.
- The order is team, group, competition, venue, schedule, player, match, event type, match event.
- After right clicking press “Import/Export Data”.
- This brings up a menu, under General make sure you select the correct csv file, under Options make sure the slider for Header is turned on. This allows us to import tables with headers on, and to make sure the headers do not get entered as data as well.

Testing

- To do the testing you want to open another query tool in your database (previously explained how to do this).
- You then want to copy this code:

```
--1. Listing all students, who play for a particular department (i.e. Cohort 4 group).
SELECT
  p.player_name,
  g.group_name
```

```
FROM Player p
JOIN "group" g ON p.group_id = g.group_id   --joining the group
table to the player table using the primary/foreign key group_id
WHERE g.group_name = 'Cohort 4';

/*2. Listing all fixtures for a specific date (i.e. 29th of October
2022 and including
team names and venues).*/
SELECT
   m.match_id,
   s.match_date,
   t1.team_name AS home_team,
   t2.team_name AS away_team,
   v.venue_name
FROM Match m
--joining teams, schedule and venue on match by the
primary/foreign key relating the 2 tables together.
JOIN Schedule s ON m.schedule_id = s.schedule_id
JOIN Team t1 ON m.home_team_id = t1.team_id
JOIN Team t2 ON m.away_team_id = t2.team_id
JOIN Venue v ON m.venue_id = v.venue_id
WHERE s.match_date = '2022-10-29';

--3. Listing all the players who have scored more than 2 goals
SELECT
   p.player_name,
   COUNT(*) AS goals_scored  --counts goals scored
FROM Player p
/*joining match event on player and event type on match event
we can now see which player did what event*/
JOIN Match_Event me ON p.player_id = me.player_id
JOIN Event_Type et ON me.event_type_id = et.event_type_id
WHERE et.event_type = 'Goal'  --selects the event type goals
GROUP BY p.player_name
HAVING COUNT(*) > 2;
```

```
--4. Listing the number of cards (yellow and red) per team.
SELECT
   t.team_name,
   COUNT(*) AS total_cards  --counts total number of cards
FROM Team t
/*joining player on team, match event on player, event type on
match event
allows us to see which players got cards and which team they
play for*/
JOIN Player p ON t.team_id = p.team_id
JOIN Match_Event me ON p.player_id = me.player_id
JOIN Event_Type et ON me.event_type_id = et.event_type_id
WHERE et.event_type IN ('Yellow Card', 'Red Card')  --selects all
cards
GROUP BY t.team_name
ORDER BY total_cards DESC;

--5. Return the games that are going to be played (friendly
matches).
SELECT
   m.match_id,
   s.match_date,
   t1.team_name AS home_team,
   t2.team_name AS away_team
FROM Match m
/*joining competition, schedule and team onto match allows
us to
see which matches, when they are played and who is playing
for all the friendly games*/
JOIN Competition c ON m.competition_id = c.competition_id
JOIN Schedule s ON m.schedule_id = s.schedule_id
JOIN Team t1 ON m.home_team_id = t1.team_id
JOIN Team t2 ON m.away_team_id = t2.team_id
```

```
WHERE c.competition_type = 'Friendly';  --selects only the
friendly games

/*6. The table displays the team's name and the points earned
during the
tournament.*/
WITH goals AS (
  SELECT
    m.match_id,
    m.home_team_id,
    m.away_team_id,

            --finds the score of each game
    SUM(CASE
      WHEN p.team_id = m.home_team_id THEN 1 ELSE 0
    END) AS home_goals,

    SUM(CASE
      WHEN p.team_id = m.away_team_id THEN 1 ELSE 0
    END) AS away_goals

  FROM Match m
  JOIN Match_Event me ON m.match_id = me.match_id
  JOIN Player p ON me.player_id = p.player_id
  JOIN Event_Type et ON me.event_type_id = et.event_type_id
  WHERE et.event_type = 'Goal'
  GROUP BY m.match_id, m.home_team_id, m.away_team_id
),

--calculates points based on which team has more goals or if
they have the same goals
points AS (
  SELECT
    home_team_id AS team_id,
    CASE
```

```
        WHEN home_goals > away_goals THEN 3
        WHEN home_goals = away_goals THEN 1
        ELSE 0
      END AS pts
    FROM goals

    UNION ALL

    SELECT
      away_team_id AS team_id,
      CASE
        WHEN away_goals > home_goals THEN 3
        WHEN away_goals = home_goals THEN 1
        ELSE 0
      END AS pts
    FROM goals
)

SELECT
    t.team_name,
    SUM(p.pts) AS total_points
FROM points p
JOIN Team t ON p.team_id = t.team_id
GROUP BY t.team_name
ORDER BY total_points DESC;

/*7. For each team, present the distribution of goals scored
and conceded in each
half of the match.*/
SELECT
    t.team_name,

    -- Adds together the goals scored in the first half
    SUM(CASE
```

```
    WHEN p.team_id = t.team_id --when team_id is the same for the event and the player it means the goal is scored by that team
    AND me.minute <= 45 THEN 1 ELSE 0 --minute<=45 means its in the first half
  END) AS goals_scored_1st_half,

  -- Adds together the goals scored in the second half
  SUM(CASE
    WHEN p.team_id = t.team_id --when team_id is the same for the event and the player it means the goal is scored by that team
    AND me.minute > 45 THEN 1 ELSE 0 --minute>45 means its in the second half
  END) AS goals_scored_2nd_half,

  -- Goals conceded (1st half)
  SUM(CASE
    WHEN p.team_id != t.team_id --when team_id is not the same for the event and the player it means the goal is conceded by that team
    AND me.minute <= 45 THEN 1 ELSE 0 --minute<=45 means its in the first half
  END) AS goals_conceded_1st_half,

  -- Goals conceded (2nd half)
  SUM(CASE
    WHEN p.team_id != t.team_id --when team_id is not the same for the event and the player it means the goal is conceded by that team
    AND me.minute > 45 THEN 1 ELSE 0 --minute>45 means its in the second half
  END) AS goals_conceded_2nd_half
```

--Joining together tables that allow us to use team_id to see if the goal is scored or conceded
FROM Team t
JOIN Match m ON t.team_id IN (m.home_team_id, m.away_team_id)
JOIN Match_Event me ON m.match_id = me.match_id
JOIN Player p ON me.player_id = p.player_id
JOIN Event_Type et ON me.event_type_id = et.event_type_id

WHERE et.event_type = 'Goal'

GROUP BY t.team_name;
END OF SCRIPT

- You want to highlight the test you want to do then run that to show the result for that test.
- Running it all at once will only show the result for the last test.

ERD

- If you want PostgreSQL to create an ERD for you right click on your database and click ERD for Database.